

Student Record Management System

Python Essentials – Evaluated Course Project

Submitted by: **Tanmay Mishra**
Programme: B.Tech CSE (Core), First Year
Institution: VIT Bhopal
Course Instructor: Dr. D. Lakshmi
Date: September 2026

1. Introduction

This project applies the core concepts covered in the Python Essentials course — variables, loops, conditional statements, functions, dictionaries, lists, and file handling — to build a functional, modular, command-line application. The chosen application, a Student Record Management System, allows a user to add, view, search, delete, and analyze student records, with all data persisted to a local file between sessions.

2. Problem Statement

Instructors and small institutions often need a lightweight way to record and track basic student performance data without relying on complex, dependency-heavy database systems or external software. Manual record-keeping on paper is error-prone and difficult to search, while spreadsheet tools require additional software and are not aligned with the programming concepts being learned in this course. This project addresses that gap by providing a simple, dependency-free, terminal-based application that lets a user create, view, search, delete, and analyze student records using only Python's built-in features, with results persisted between sessions through basic file storage.

3. Functional Requirements

The system is organized into three major functional modules:

- **Student Management** — add, view, search, and delete student records (roll number, name, marks).
- **File Persistence** — automatically save records to `students.txt` and reload them the next time the program runs.
- **Reports & Analytics** — compute the total student count, average marks, the topper, and a rank-based (relative) grade for every student.

Input / Output structure:

- **Input:** roll number, name, and marks entered via the terminal; a numeric menu choice (1–6).
- **Output:** formatted record listings, search/delete confirmations, and a summary report printed to the terminal.

Workflow:

The user is presented with a numbered menu on program start. Based on the selected option, control is passed to the relevant module (student management, file handling, or reporting), after which the menu is redisplayed. This repeats until the user selects Exit.

4. Non-Functional Requirements

Performance: All operations run in-memory on a Python list, so add/view/search/delete complete instantly even with a few hundred records; the only I/O-bound step is writing to `students.txt`, which is done only when data actually changes.

Reliability: Every add or delete operation immediately triggers a save to disk, so a completed operation is never lost even if the program is closed right afterward.

Usability: A clear, numbered menu with descriptive prompts allows the system to be operated without needing to read the underlying code.

Maintainability: The code is split into four independent modules (main, student_manager, file_handler, reports), each responsible for a single concern, so a change to one feature does not require touching unrelated code.

Error Handling: Lookups and empty-state operations are explicitly checked (e.g. “No records found”, “No student found with that roll number”) rather than allowing the program to fail with an unhandled exception.

Scalability: Because each student is stored as a dictionary, additional fields (e.g. subject-wise marks, attendance) can be added with minimal structural change to the rest of the system.

Resource Efficiency: The system relies only on Python’s built-in os module and standard file I/O — no external libraries or installation steps are required.

5. System Architecture

The system follows a modular, function-based architecture. `main.py` acts as the controller: it owns the in-memory list of student records and delegates specific responsibilities to three collaborator modules. Data is passed between modules explicitly as function parameters and return values, rather than through shared global state, which keeps each module independently understandable and testable.

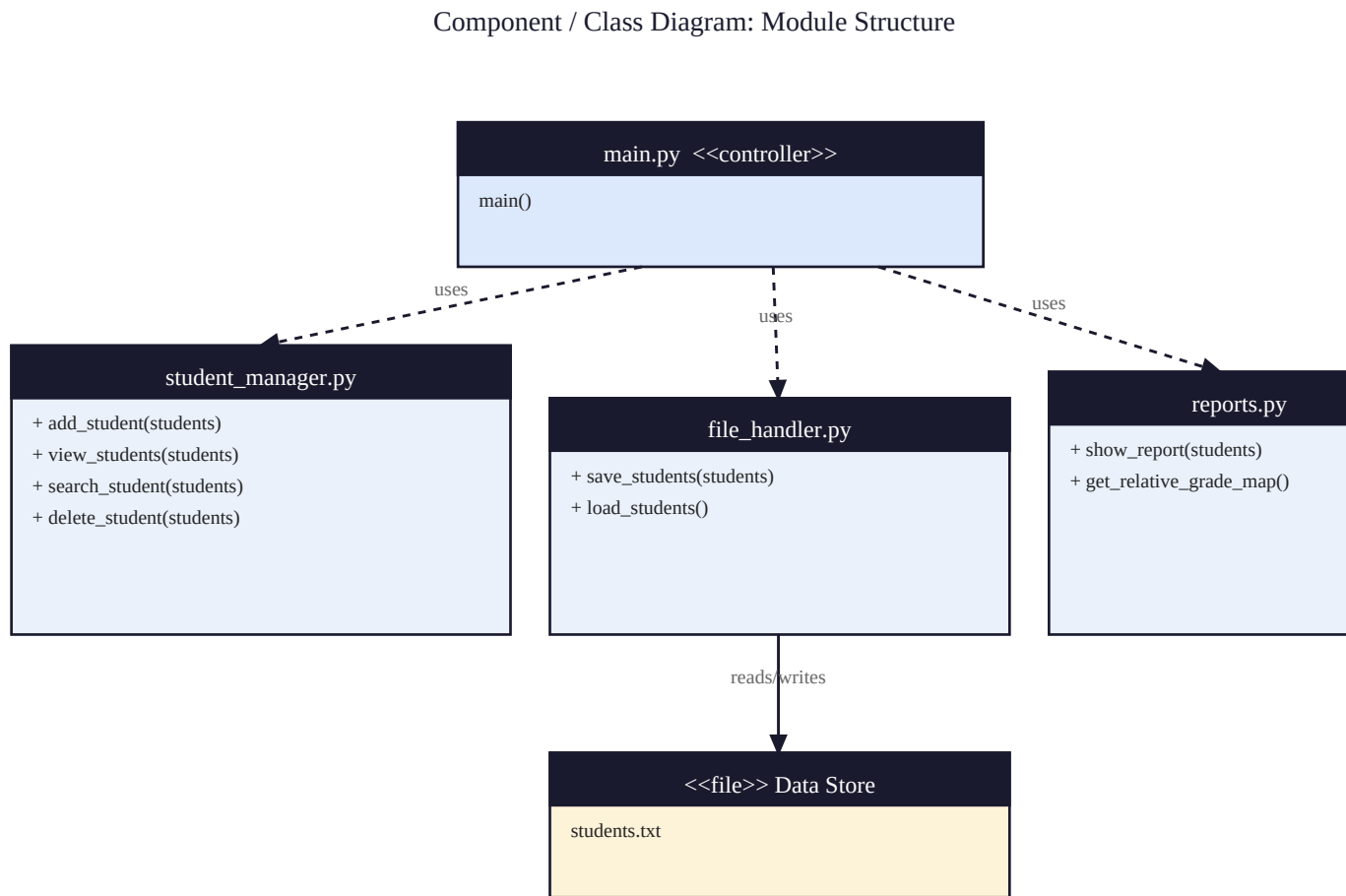


Figure 1: Component diagram showing `main.py` delegating to `student_manager.py`, `file_handler.py`, and `reports.py`, with `file_handler.py` reading from and writing to `students.txt`.

6. Design Diagrams

6.1 Use Case Diagram

Use Case Diagram: Student Record Management System

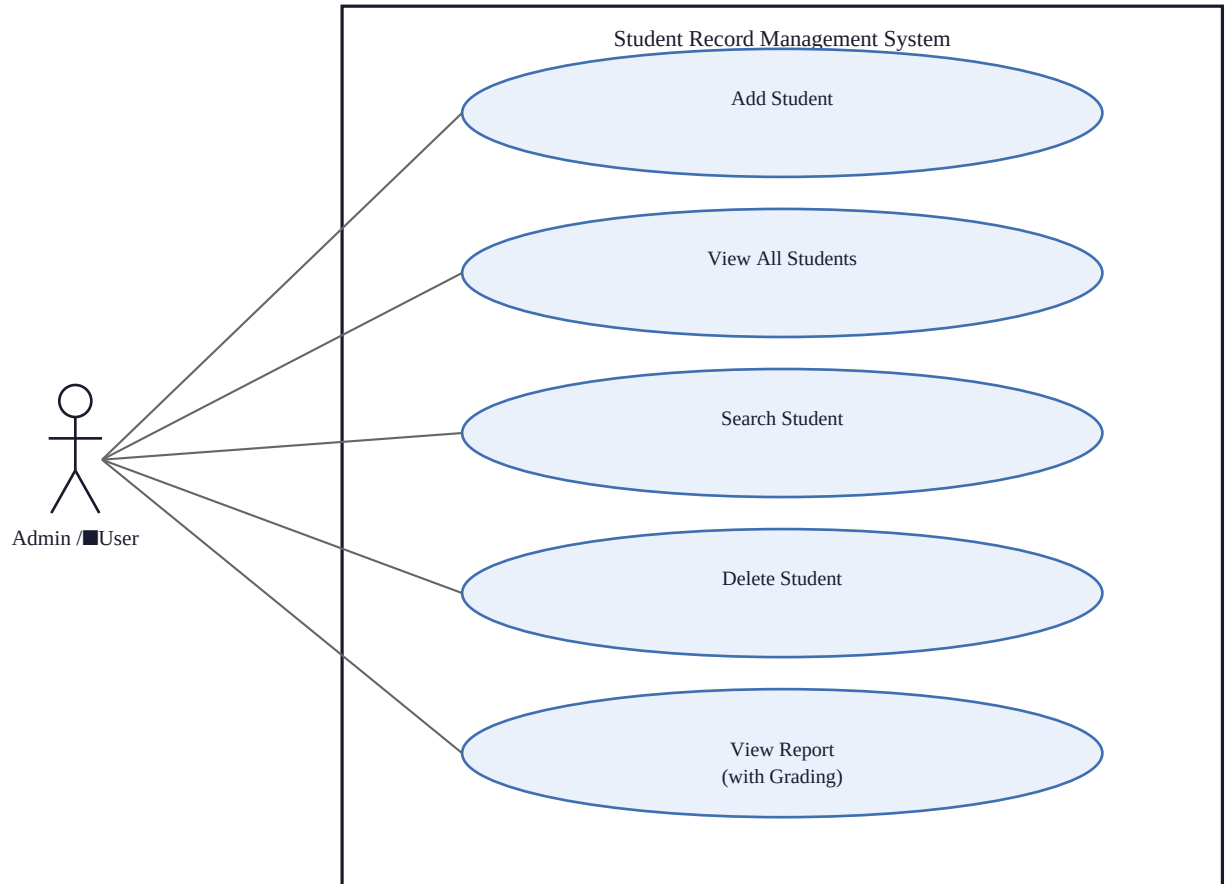


Figure 2: The Admin/User actor interacts with five use cases exposed by the system.

6.2 Workflow / Process Flow Diagram

System Workflow / Process Flow Diagram

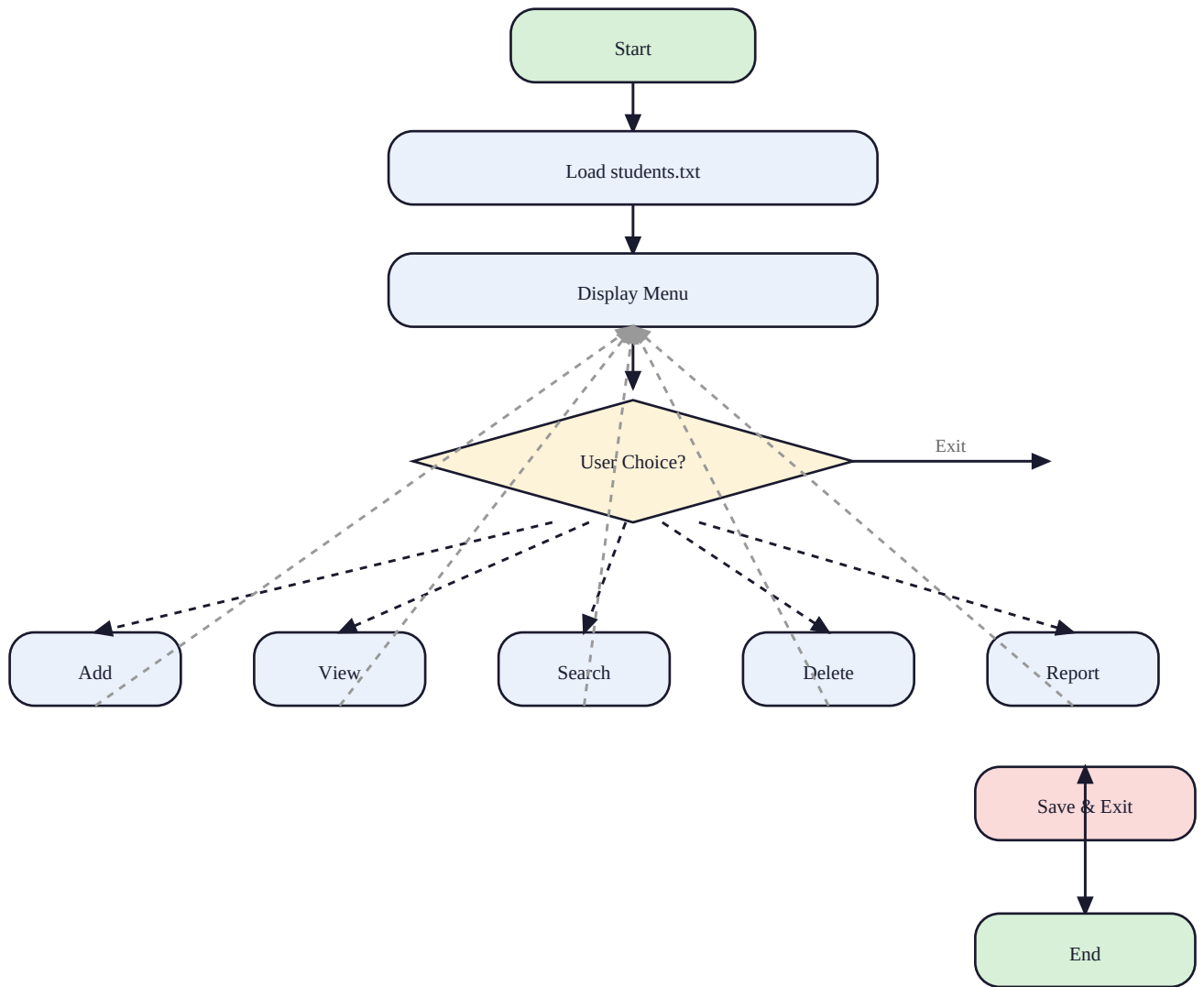


Figure 3: Program flow from start-up (loading existing data) through the menu loop to exit (saving data).

6.3 Sequence Diagram – Add Student Flow

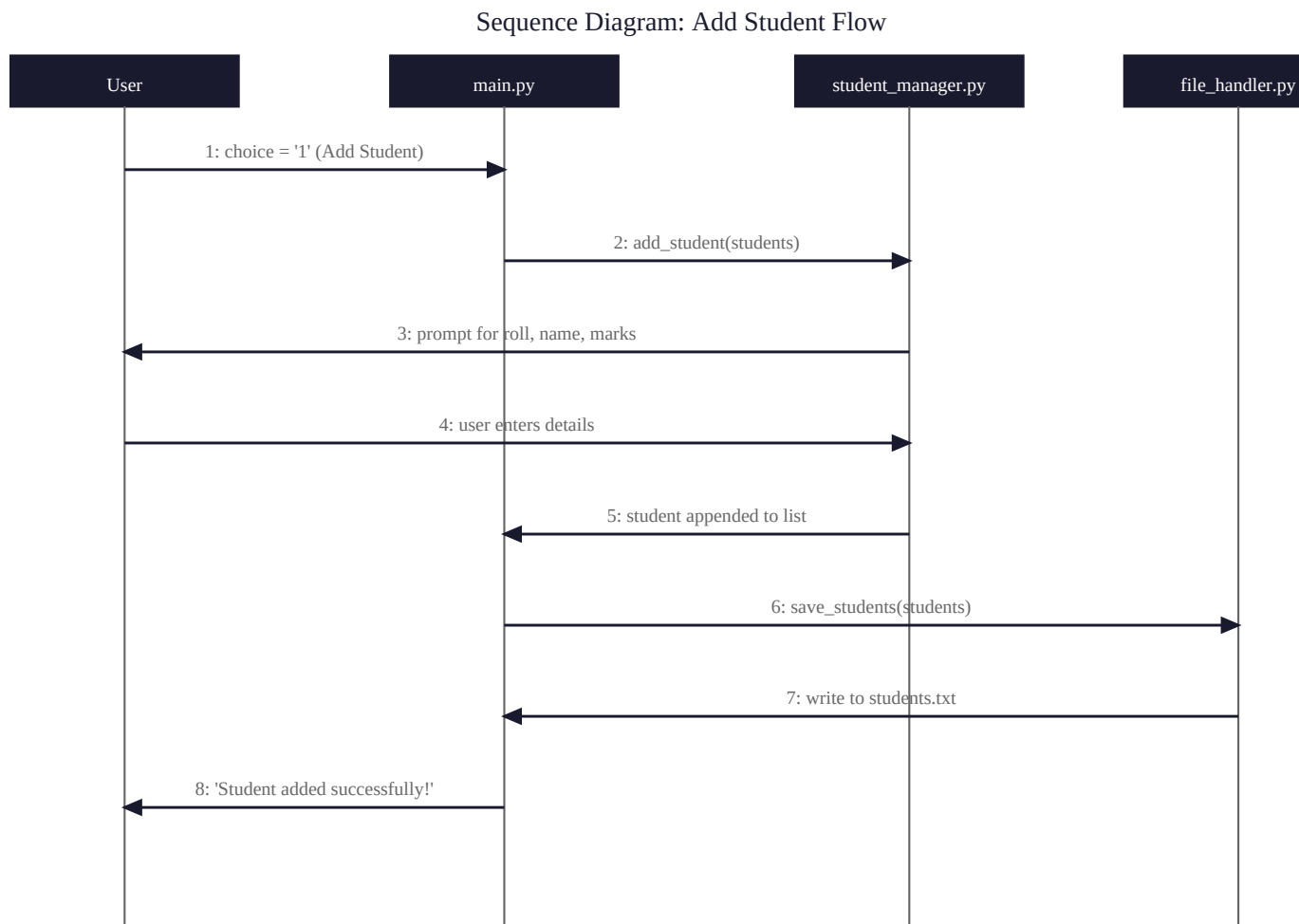


Figure 4: Message sequence between the User, main.py, student_manager.py, and file_handler.py when adding a new student record.

7. Design Decisions & Rationale

- **Dictionaries over custom classes:** each student is represented as a dictionary rather than a custom class, aligning with the course scope (object-oriented programming had not yet been introduced).
- **Plain-text storage over a database:** a comma-separated text file was chosen over an external database for transparency, human readability, and to avoid dependencies beyond the standard library.
- **Passing data explicitly:** the student list is passed into every function as a parameter and returned where a new list is built (e.g. `load_students()`), instead of relying on global variables, so each module remains independent and reusable.
- **Rank-based grading via a dictionary map:** grades are computed once per unique mark value and stored in a dictionary, so students who scored identical marks are guaranteed identical grades without recomputing the ranking for every student individually.

8. Implementation Details

student_manager.py — core CRUD operations:

- `add_student(students)`: collects roll, name, and marks via `input()`, builds a dictionary, and appends it to the shared list.
- `view_students(students)`: iterates over the list and prints each record, or a “No records found” message if empty.
- `search_student(students) / delete_student(students)`: use a boolean flag pattern to track whether a matching roll number was found during the loop.

file_handler.py — persistence:

- `save_students(students)`: writes each record as one comma-separated line.
- `load_students()`: checks whether the file exists using `os.path.exists()` before attempting to read it, avoiding a crash on the very first run.

reports.py — analytics:

- `show_report(students)`: computes total count, average marks, and the topper using a running-comparison loop.
- `get_relative_grade_map(students)`: collects unique mark values, sorts them in descending order, and maps each to a grade label (S, A, B, C, D, E, F) based on rank.

9. Screenshots / Results

[Insert terminal screenshots here showing: the main menu, adding a student, viewing all records, searching a valid and invalid roll number, deleting a record, and the final report with grades.]

10. Testing Approach

The system was tested manually by exercising both expected and edge-case inputs at each menu option:

- Adding multiple records, including students with identical marks, to verify the report and grading logic handle ties correctly.
- Searching and deleting with both a valid roll number and a roll number that does not exist, to confirm both success and “not found” paths behave correctly.
- Restarting the program after adding records to confirm data persistence via `students.txt`.
- Viewing the report with an empty student list to confirm the “No records found” guard clause prevents a crash.

Formal input validation (e.g. rejecting non-numeric marks) was scoped out of this submission to keep the implementation aligned with the essentials-level concepts covered in the course; this is noted as a future enhancement below.

11. Challenges Faced

- The project began as a single-file script; restructuring it into separate modules required understanding how Python passes data between files using parameters and return values, since modules no longer share a single global list automatically.
- An `ImportError` occurred while wiring modules together, eventually traced to two overlapping files (`main.py` and `main1.py`) causing confusion about which file was actually being edited and run.
- A missing closing parenthesis in `reports.py` caused the program to stop silently partway through a function, with no visible error message; this was diagnosed by inserting a temporary debug print statement to isolate exactly where execution stopped.
- This experience highlighted the importance of verifying file identity directly through the terminal (e.g. `pwd`, `find`) rather than relying solely on editor tabs, which can be misleading when multiple similarly named files exist.

12. Learnings & Key Takeaways

- Practical experience splitting a single script into cooperating modules using imports, parameters, and return values.
- A concrete understanding of mutability: functions that modify an existing list (append/remove) do not need to return it, while functions that build a new list from scratch must return it explicitly.
- A debugging strategy for silent failures: inserting temporary print statements as “checkpoints” to narrow down exactly where a program stops behaving as expected.

- The value of verifying assumptions directly (which file is actually running, from which folder) rather than trusting the editor interface alone.

13. Future Enhancements

- Add input validation for numeric fields and empty inputs (e.g. rejecting non-numeric marks).
- Support subject-wise marks instead of a single aggregate mark per student.
- Export the generated report to a CSV or PDF file.
- Migrate storage from a plain text file to SQLite for more structured querying.
- Add a graphical (GUI) or web-based front end.

14. References

- Python Software Foundation, Python 3 Official Documentation — docs.python.org
- VIT VITyarthi, “Build Your Own Project — General Instructions & Submission Guidelines”